

`grind`: An SMT-Inspired Tactic for Lean 4 (Short Paper – System Description)

Kim Morrison[1(✉)] and Leonardo de Moura[1,2]

[1] Lean FRO, Washington, USA
{kim,leo}@lean-fro.org
[2] Amazon Web Services, Washington, USA
leomoura@amazon.com

Abstract. We describe `grind`, an SMT-inspired proof automation tactic for Lean 4. Unlike hammer-style tools that translate proof obligations into external logics and invoke external ATP and SMT solvers, `grind` works natively in the Calculus of Inductive Constructions, producing kernel-checkable proof terms without any soundness compromises. At its core, `grind` combines congruence closure for dependent type theory with E-matching and a suite of satellite solvers for linear integer arithmetic, linear arithmetic over ordered modules, polynomial equations over commutative (semi)rings, and associative-commutative operators. Each satellite solver is parameterized by Lean's typeclasses, enabling it to operate over any type that implements the appropriate algebraic interface—not just hardcoded numeric types. Users can extend `grind` with new theory solvers through a plugin API, control theorem instantiation through instantiation constraints on E-matching patterns, and inspect the internal state through an interactive DSL. An annotation system, including automatic pattern selection, enables libraries to declare how their theorems should be used by `grind`. The tactic is distributed as part of Lean and is used extensively in Lean's Mathlib [19] and CSLib [2] libraries.

1 Introduction

Proof assistants based on dependent type theory—Coq [18] and Lean 4 [12]—provide strong guarantees: every accepted proof is checked by a small trusted kernel. The price is that constructing proofs often requires substantial manual effort.

Two families of automation dominate current practice. *Rewriting-based* tactics such as `simp` apply oriented equations exhaustively. *Hammer-style* tools translate goals into first-order or higher-order logic, invoke external provers, and reconstruct proofs internally. Sledgehammer [14] pioneered this approach for Isabelle/HOL; CoqHammer [3] adapted it for Coq; lean-auto [15] and LeanHammer [20] bring it to Lean 4. Translation introduces a gap: not every feature of dependent type theory has a counterpart in the target logic, and reconstruction can fail even when the external prover succeeds.

© The Author(s) 2026
A. Biere et al. (Eds.): IJCAR 2026, LNAI 16688, pp. 106–114, 2026.
https://doi.org/10.1007/978-3-032-32589-1_7

grind takes a different approach. Inspired by the architecture of SMT solvers [1,11], it combines congruence closure, E-matching, and theory solvers into a single tactic operating directly on Lean expressions in the Calculus of Inductive Constructions. There is no translation step and no external tool: every proof term is checked by Lean's kernel. The congruence closure procedure handles the full Calculus of Inductive Constructions, extending earlier work [17].

Each satellite solver in grind is parameterized by Lean's typeclass mechanism. The linear arithmetic solver operates over any type equipped with a module structure over a semiring, and becomes incrementally more powerful over a ring or field. This design enables grind to handle the rich algebraic hierarchy found in mathematical libraries, without hardcoding specific types.

grind is open source,[1] distributed as part of Lean 4 with no external dependencies, and has been available since Lean 4.22 (August 2025). We recommend the reader consult the reference manual.[2] The implementation comprises approximately 40 000 lines of Lean.

Contributions. This paper describes grind with a focus on the following contributions. (1) *Native operation in the CIC.* grind combines congruence closure, E-matching, and theory solvers in a single framework that operates directly on Lean expressions—including dependent types, dependent pattern matching, universe polymorphism, and proof irrelevance—producing kernel-checkable proof terms without translation to an external logic. (2) *Typeclass-parametrized solvers.* Each satellite solver is parameterized by Lean typeclasses, activating automatically for any type with the required algebraic structure. This contrasts with traditional SMT solvers that hardcode specific types. (3) *Annotation and instantiation control.* An annotation system with automatic pattern selection lets library authors declare how their theorems should be used by grind, and a constraint language on E-matching patterns restricts when theorems are instantiated (Sect. 2). (4) *Extensibility.* All built-in solvers are implemented through the same plugin API available to users, who can register new theory solvers and constraint propagators without leaving Lean. (5) *Proof stability and diagnostics.* grind? outputs self-contained invocations listing the theorems actually used, an interactive mode (grind =>) exposes the solver's execution step by step, and comprehensive failure diagnostics report equivalence classes, satellite solver states, and resource limits reached.

2 grind by Example

All examples here can be explored interactively at https://live.lean-lang.org/.

Congruence closure. At the core of grind is a congruence closure algorithm for dependent type theory [17]. It maintains equivalence classes over the terms. When two terms are known to be equal, congruence closure automatically deduces equalities between larger expressions built from them.

[1] https://github.com/leanprover/lean4/tree/master/src/Lean/Meta/Tactic/Grind.
[2] https://lean-lang.org/doc/reference/latest/The--grind--tactic.

```
example (f : Nat → Nat) (h : a = b) : f (f b) = f (f a) := by grind
```

Here `grind` notices the `a = b` hypothesis, applies congruence to obtain `f a = f b`, and then concludes `f (f a) = f (f b)`. A contradiction is detected when the equivalence classes containing `True` and `False` are merged. Before adding terms to the E-graph, `grind` normalizes them: β-reduction, canonicalization of instances and implicit arguments, and hash-consing so that structural equality reduces to pointer equality.

E-Matching and Annotations. While the Lean `simp` tactic uses theorems as rewriting rules, `grind` instantiates theorems using E-matching. Each theorem annotated for `grind` carries one or more *patterns*. Whenever the E-graph contains terms that match a pattern modulo known equalities, `grind` instantiates the theorem and asserts the result as a new fact.

```
@[grind =] theorem fg : f (g x) = x
example : f a = b → a = g c → b = c := by grind
```

The attribute `[grind =]` selects the left-hand side of the equation, `f (g x)`, as the E-matching pattern. Although the goal does not contain a literal instance of `f (g x)`, the hypothesis `a = g c` allows `grind` to match `f a` by substituting `a` with `g c` modulo the known equality.

Library authors typically do not want to specify patterns manually for every theorem. `grind` provides a family of annotations that capture commonly used idioms. For equational theorems, `[grind =]` selects the left-hand side as the pattern, `[grind _=_]` selects both sides, and the bare `[grind]` attribute invokes an algorithm that traverses the conclusion and hypotheses left to right, collecting the minimal set of indexable subexpressions—those with a constant at the head—that cover all universally quantified variables. For implications, `[grind →]` extracts patterns from the hypotheses (forward reasoning) and `[grind ←]` from the conclusion (backward reasoning).

As an example of forward reasoning, consider annotating transitivity and symmetry lemmas for a relation `R`:

```
def R : Nat → Nat → Prop
@[grind →] theorem Rtrans : R x y → R y z → R x z
@[grind →] theorem Rsymm  : R x y → R y x
example : R a b → R c b → R d c → R a d := by grind
```

When `grind` discovers that `R a b` and `R c b` hold, E-matching triggers `Rsymm` to derive `R b c`, then `Rtrans` chains the results. For theorems that do not fit any built-in idiom, the `grind_pattern` command allows manual specification of patterns, including *multi-patterns* that require all components to match simultaneously before the theorem is instantiated.

Instantiation Constraints. Consider the theorem stating that left-shift distributes over addition in the exponent:

```
theorem shiftLeft_add (m n : Nat) : ∀ k, m ≪≪≪ (n + k) = (m ≪≪≪ n) ≪≪≪ k
grind_pattern shiftLeft_add => m ≪≪≪ (n + k) where m ≠ 0
```

The constraint `m ≠ 0` prevents instantiation when `m` is definitionally equal to zero, since the separate theorem `zero_shiftLeft` (stating $0 \lll n = 0$) already covers that case. Without the constraint, `grind` would generate redundant instances. Other available constraints include generation bounds, instance count limits, structural conditions on matched terms (e.g., requiring a term to be ground or a value), and guards that postpone instantiation until a proposition is known to hold.

Case Splitting and Inductive Predicates. `grind` does not case-split over arbitrary Boolean variables like a SAT solver. Instead, it follows structure: splitting on `if-then-else` expressions, `match` expressions, and inductive predicates. For inductive predicates, the annotation `[grind]` enables both case analysis on hypotheses and introduction using the constructors. As an example, we define big-step operational semantics for a simple imperative language.

```
attribute [grind] BigStep

theorem if_iff {B S T s t} : (ifThenElse B S T, s) ==>
    t ↔ (B s ∧ (S, s) ==> t) ∨ (¬ B s ∧ (T, s) ==> t) := by grind
```

Here `grind` case-splits on the `BigStep` constructors to decompose the hypothesis, propagates the condition `B s` or its negation into the congruence closure module, and applies the `if_true` and `if_false` constructors to close each branch.

Dependent Pattern Matching. When match-expressions involve indexed inductive families, the Calculus of Inductive Constructions requires unifying type indices and inserting cast operations—features with no counterpart in first-order or higher-order logic. Consider a function on length-indexed vectors:

```
inductive Vec (α : Type u) : Nat → Type u
  | nil : Vec α 0
  | cons : α → Vec α n → Vec α (n+1)

@[grind =] def Vec.head : Vec α (n+1) → α
  | .cons a _ => a

example (as bs : Vec Int (n+1)) : as.head = bs.head
    → (match as, bs with
       | .cons a _, .cons b _ => a + b) = 2 * as.head := by grind
```

Because the index is `n+1`, both arguments must be `.cons`—the `.nil` case is ruled out by the type—so `Vec.head` and the inline `match` each have a single branch instead of two and four respectively. When `grind` splits on the `match`, it generates heterogeneous equalities and cast operations that the E-graph tracks automatically. With only one case surviving, the E-graph learns `as = .cons a as'` and `bs = .cons b bs'`; E-matching then instantiates the equation for `Vec.head`, yielding `as.head = a` and `bs.head = b`. Combined with the hypothesis `as.head = bs.head`, the linear arithmetic solver closes the goal.

3 Typeclass-Parametrized Solvers

SMT solvers such as CVC5 [1] and Z3 [11] ship built-in theories for a fixed set of types—typically the integers, rationals, and reals, with fixed signatures for $+$, $\times$, $\leq$. Extending them to new types requires modifications to the solver's source code. In `grind`, each satellite solver is parameterized by Lean typeclasses. The solver activates automatically for any type that provides the required instances—no source-code changes are needed.

The Ring Solver. The ring solver reduces polynomial equalities using Gröbner bases. It activates for any type that is at least a commutative semiring, with extensions for commutative rings and fields. The following example derives the value of the fourth power sum from the first three:

```
example [CommRing α] (a b c : α) :
    a + b + c = 3 →
    a^2 + b^2 + c^2 = 5 →
    a^3 + b^3 + c^3 = 7 →
    a^4 + b^4 + c^4 = 9 := by grind
```

Because the solver is parameterized by a typeclass rather than hardcoded for a specific type, the same reasoning applies to any commutative ring. When the ring has a known characteristic p, the solver performs coefficient arithmetic modulo p via the IsCharP typeclass. For example, `BitVec 8` is a commutative ring of characteristic 256, and `grind` handles it without any bitvector-specific theory:

```
example (x : BitVec 8) : (x - 16) * (x + 16) = x^2 := by grind
```

This holds because $16 \times 16 = 256 \equiv 0 \pmod{256}$. The ring solver also handles commutative semirings—where subtraction is not available—by constructing the Grothendieck completion to obtain a ring:

```
example [CommSemiring α] [AddRightCancel α] (x y : α) :
    x^2*y = 1 → x*y^2 = y → y*x = 1 := by grind
```

Linear Arithmetic and Orders. The linear arithmetic solver handles any type equipped with a `NatModule` (an additive commutative monoid with scalar multiplication by natural numbers) or an `IntModule` (an additive commutative group with scalar multiplication by integers). It extends naturally to ordered settings—supporting preorders, partial orders, and linear orders—and has further extensions for semirings, rings, and fields. A second solver, `lia`, implements a CutSat-style [8] decision procedure for linear integer arithmetic. It operates over any type that admits an injection into $\mathbb{Z}$ via the `ToInt` typeclass. For example, `UInt32` maps into $[0, 2^{32})$:

```
example (a b : UInt32) : a ≤ 2 → b ≤ 3 → a + b ≤ 5 := by grind
```

A dedicated solver for partial and linear orders propagates transitivity, antisymmetry, and totality. It activates for any type with at least a preorder:

```
example [LE α] [Std.IsLinearPreorder α] (a b c d : α) :
```

```
a ≤ b → ¬ (c ≤ b) → ¬ (d ≤ c) → a ≤ d := by grind
```

Theory Combination. The satellite solvers do not operate in isolation. They share information through the congruence closure core: when one solver reports an equality, all solvers see it. The following example requires the ring solver and congruence closure to cooperate. The typeclass `NoNatZeroDivisors` provides the property that $k \neq 0 \wedge k \cdot a = 0 \implies a = 0$, which the ring solver uses for equality propagation:

```
example [CommRing α] [NoNatZeroDivisors α]
    (a b c : α) (f : α → Nat) :
    a + b + c = 3 → a^2 + b^2 + c^2 = 5 → a^3 + b^3 + c^3 = 7 →
    f (a^4 + b^4) + f (9 - c^4) ≠ 1 := by grind
```

Here the ring solver derives $a^4 + b^4 = 9 - c^4$. Congruence closure then deduces `f (a^4 + b^4) = f (9 - c^4)`, reducing the goal to $2 \cdot f(9 - c^4) \neq 1$, which is discharged by the linear integer arithmetic solver.

Extensibility. Built-in solvers are implemented through the same plugin interface that is available to users. A solver extension declares an *internalizer* (called when new terms enter the E-graph), and *equality handlers* (called when new equalities and disequalities are discovered). Because the interface is defined in Lean, users can add domain-specific solvers without leaving the language and without any privileged access to `grind`'s internals.

4 Interactive Mode and Diagnostics

When `grind` fails, or when a user wants to understand or control its behavior, the syntax `grind =>` exposes a domain-specific language for stepping through the solver's execution. Users can invoke individual solver steps, trigger E-matching, perform case splits, inspect equivalence classes, and add new hypotheses—all inside Lean, freely mixing `grind` steps with ordinary tactic proofs.

`grind?`] outputs a self-contained tactic script containing the exact steps used to close the goal. The generated script does not depend on the annotations from the environment, improving proof stability across library updates. In the following example, `grind?` generated a script with two steps. The first step instantiates only the theorem `trig_identity`, and the second applies the ring solver.

```
example : (cos x + sin x)^2 = 2 * cos x * sin x + 1 := by
  grind =>
    instantiate only [trig_identity]
    ring
```

5 Related Work

External Provers. Sledgehammer [14] pioneered the hammer approach for Isabelle/HOL, translating goals to first-order logic and invoking external ATPs

(E, Vampire) and SMT solvers (CVC5, Z3). CoqHammer [3] adapted this for Coq. In Lean 4, lean-auto [15] translates goals to monomorphic higher-order logic and dispatches to external ATPs, while LeanHammer [20] adds neural premise selection. lean-smt [10] and SMTCoq [4] integrate external SMT solvers with Lean 4 and Coq respectively, reconstructing proofs from solver certificates. These tools benefit from mature premise selection and highly optimized solver backends, making them more scalable when large numbers of lemmas are available. The cost is a translation gap: not every feature of the Calculus of Inductive Constructions has a counterpart in the target logic, reconstruction can fail even when the external prover succeeds, and the round-trip adds latency. `grind` avoids all three costs by operating directly on Lean expressions and producing kernel-checkable proof terms incrementally. Because it runs inside the Lean elaborator, verification frameworks built in Lean—such as Aeneas [6] and Velvet [5]—can share state across `grind` calls: E-graph contents, resolved instances, and asserted facts carry over when multiple verification conditions share a common proof context.

In Lean 4, the `simp` tactic performs equational rewriting. The `omega` tactic decides Presburger arithmetic over `Nat` and `Int`. Aesop [9] performs best-first search over tactic scripts. Canonical [13] is a type inhabitation solver that exhaustively searches for proof terms in a canonical form. The `egg` tactic [16] uses equality saturation to explore the space of equivalent terms via e-graphs, similar to `grind`'s directed congruence closure. `grind` complements these: it combines theory reasoning with E-matching in a single framework, handles algebraic structures through typeclasses, and supports user extensibility through the plugin API and the constraint propagator interface.

6 Evaluation and Conclusion

`grind` was released in August 2025 and has been rapidly adopted. As of February 2026, Mathlib uses `grind` approximately 3 400 times across 2 400 theorems. Of these, around 1 200 were initially authored using `grind`, while the other 1 200 were refactored after `grind` became available. These refactors averaged a reduction of 28% in lines of code, with ∼50 theorems seeing proofs reduced by 10 lines or more.[3] CSLib [2], the Prime Number Theorem[4] project and Terence Tao's Analysis I[5] repository each also uses `grind` around 500 times.

The most immediate opportunities lie in scaling premise selection to large unannotated libraries. We already have a configurable premise selection engine (`grind +suggestions`), which uses trigger-based relevance filtering inspired by [7] to rank unannotated library theorems by relevance to the current goal, but it needs tuning for Lean's libraries, and we anticipate integrating external neural premise selection engines. We also plan to scale better when very large (>1000)

[3] https://github.com/leanprover/lean4/blob/master/doc/examples/IJCAR2026/analyze_grind_loc.py.
[4] https://github.com/AlexKontorovich/PrimeNumberTheoremAnd.
[5] https://github.com/teorth/analysis.

numbers of lemmas are available for instantiation, and have initial linting tools to help users design good annotations.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Barbosa, H., et al.: Cvc5: a versatile and industrial-strength SMT solver. Presented at the (2022). https://doi.org/10.1007/978-3-030-99524-9_24
2. Barrett, C., et al.: CSLib: the Lean computer science library. arXiv preprint arXiv:2602.04846 (2026). https://doi.org/10.48550/arXiv.2602.04846
3. Czajka, Ł, Kaliszyk, C.: Hammer for Coq: automation for dependent type theory. J. Autom. Reason. **61**, 423–453 (2018)
4. Ekici, B., et al.: SMTCoq: a plug-in for integrating SMT solvers into Coq. In: Majumdar, R., Kunčak, V. (eds.) CAV 2017. LNCS, vol. 10427, pp. 126–133. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-63390-9_7
5. Gladshtein, V., Pîrlea, G., Zhao, Q., Kurin, V., Sergey, I.: Foundational multimodal program verifiers. Proc. ACM Program. Lang. **10**(POPL) (2026). https://doi.org/10.1145/3776719
6. Ho, S., Protzenko, J.: Aeneas: rust verification by functional translation. Proc. ACM Program. Lang. **6**(ICFP), 711–741 (2022).https://doi.org/10.1145/3547647
7. Hoder, K., Voronkov, A.: Sine Qua non for large theory reasoning. In: Bjørner, N., Sofronie-Stokkermans, V. (eds.) CADE 2011. LNCS (LNAI), vol. 6803, pp. 299–314. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-22438-6_23
8. Jovanović, D., de Moura, L.: Cutting to the chase: solving linear integer arithmetic. J. Autom. Reason. **51**(1), 79–108 (2013). https://doi.org/10.1007/s10817-013-9281-x
9. Limperg, J., From, A.H.: Aesop: white-box best-first proof search for Lean. In: Proceeding CPP 2023, pp. 253–266. ACM (2023)
10. Mohamed, A., et al.: lean-SMT: an SMT tactic for discharging proof goals in Lean. In: Proceeding CAV 2025. LNCS, vol. 15933, pp. 197–212. Springer (2025). https://doi.org/10.1007/978-3-031-98682-6_11
11. de Moura, L., Bjørner, N.: Z3: an efficient SMT solver. In: Ramakrishnan, C.R., Rehof, J. (eds.) TACAS 2008. LNCS, vol. 4963, pp. 337–340. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-78800-3_24
12. Moura, L., Ullrich, S.: The lean 4 theorem prover and programming language. In: Platzer, A., Sutcliffe, G. (eds.) CADE 2021. LNCS (LNAI), vol. 12699, pp. 625–635. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-79876-5_37
13. Norman, C., Avigad, J.: Canonical for automated theorem proving in Lean. In: Proc. ITP 2025. LIPIcs, vol. 352, pp. 14:1–14:20. Schloss Dagstuhl – Leibniz-Zentrum für Informatik (2025). https://doi.org/10.4230/LIPIcs.ITP.2025.14
14. Paulson, L.C., Blanchette, J.C.: Three years of experience with Sledgehammer, a practical link between automatic and interactive theorem provers. In: Proceeding IWIL 2010. EPiC Series in Computing, vol. 2, pp. 1–11. EasyChair (2010). https://doi.org/10.29007/36dt
15. Qian, Y., Clune, J., Barrett, C., Avigad, J.: Lean-auto: an interface between lean 4 and automated theorem provers. In: Piskac, R., Rakamarić, Z. (eds.) Computer Aided Verification, pp. 175–196. Springer Nature Switzerland, Cham (2025)

16. Rossel, M., Schneider, R., Koehler, T., Steuwer, M., Goens, A.: Towards pen-and-paper-style equational reasoning in interactive theorem provers by equality saturation. Proc. ACM Program. Lang. **10**(POPL), 718–747 (2026). https://doi.org/10.1145/3776667
17. Selsam, D., de Moura, L.: Congruence closure in intensional type theory. In: Olivetti, N., Tiwari, A. (eds.) IJCAR 2016. LNCS (LNAI), vol. 9706, pp. 99–115. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-40229-1_8
18. The Coq Development Team: The Coq proof assistant (2024). https://coq.inria.fr/
19. The Mathlib Community: The Lean mathematical library. In: Proceeding CPP 2020, pp. 367–381. ACM (2020). https://doi.org/10.1145/3372885.3373824. https://github.com/leanprover-community/mathlib4
20. Zhu, T., Clune, J., Avigad, J., Jiang, A.Q., Welleck, S.: Premise selection for a Lean hammer (2025)

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (http://creativecommons.org/licenses/by/4.0/), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

